

Codes

Python Code (Exported from IPYNB)

```
# %% [markdown]
# # **Main Imports**

# %%
import pandas as pd
import os
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.feature_selection import mutual_info_classif
import matplotlib.pyplot as plt

# %% [markdown]
# ## *global file config*

# %%
BASE = os.getcwd()

DATASETS_DIR = "datasets"
DATASETS_PATH = os.path.join(BASE, DATASETS_DIR)

FILE_NAME = "Credit Card Defaulter Prediction.csv"
FILE_PATH = os.path.join(DATASETS_PATH, FILE_NAME)

# %% [markdown]
# ## **Part A**

# %%
from sklearn.metrics import accuracy_score, classification_report,
confusion_matrix, roc_curve, auc, ConfusionMatrixDisplay

# %%
df_a = pd.read_csv(FILE_PATH)

# %%
df_a.head()

# %%
df_a.info()

# %%
df_a.columns = df_a.columns.str.strip()

# %%
df_a['default'].value_counts()

# %%
```

```
df_a.isnull().sum()

# %%
df_a.drop(['ID'], axis=1, inplace=True)

# %%
cat_data = df_a.select_dtypes(include=['object']).columns.tolist()

# %%
cat_data

# %%
encoder = LabelEncoder()
for col in cat_data:
    df_a[col] = encoder.fit_transform(df_a[col].astype(str))

# %%
df_a.head()

# %%
X = df_a.drop(['default'], axis=1)
y = df_a['default']

# %%
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# %%
X_train = X_train.reset_index(drop=True)
y_train = y_train.reset_index(drop=True)

X_test = X_test.reset_index(drop=True)
y_test = y_test.reset_index(drop=True)

# %%
scaler = MinMaxScaler()
X_train = pd.DataFrame(scaler.fit_transform(X_train),
columns=scaler.get_feature_names_out(X_train.columns))
X_test = pd.DataFrame(scaler.transform(X_test),
columns=scaler.get_feature_names_out(X_test.columns))

# %%
feature_scorer = mutual_info_classif(X_train, y_train)
feature_scores = pd.DataFrame({'feature': X_train.columns, 'feature_score':
feature_scorer.tolist()}).sort_values(by='feature_score', ascending=False)

# %%
selected = feature_scores[feature_scores['feature_score'] > 0.01]
['feature'].tolist()

# %%
X_train = X_train[selected]
X_test = X_test[selected]
```

```

# %%
def plot_probabilities(X_test, y_test, classifier):
    try:
        pred_probs = classifier.predict_proba(X_test)[: , 1]
        title = 'Receiver Operating Characteristic (ROC) Curve | 
    {}'.format(classifier.__class__.__name__)
    except AttributeError:
        pred_probs = classifier.predict(X_test)
        title = 'Receiver Operating Characteristic (ROC) Curve | Probit Model'
    fpr, tpr, thresholds = roc_curve(y_test, pred_probs)
    roc_auc = auc(fpr, tpr)

    plt.figure()
    plt.plot(fpr, tpr, color='blue', label='ROC curve (area = 
    {:.2f})'.format(roc_auc))
    plt.plot([0, 1], [0, 1], color='red', linestyle='--')
    plt.xlim([0.0, 1.0])
    plt.ylim([0.0, 1.0])
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title(title)
    plt.legend(loc='lower right')
    plt.show()

# %% [markdown]
# ### **LogisticRegression**

# %%
from sklearn.linear_model import LogisticRegression

# %%
lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train, y_train)
y_preds_lr = lr.predict(X_test)

# %%
clf_report_lr = classification_report(y_test, y_preds_lr, output_dict=True)
clf_report_lr

# %%
plot_probabilities(X_test, y_test, lr)

# %%
# Compute the confusion matrix
conf_matrix_lr = confusion_matrix(y_test, y_preds_lr)

# Display the confusion matrix
ConfusionMatrixDisplay(confusion_matrix=conf_matrix_lr).plot()
plt.show()

# %% [markdown]
# ### **DecisionTree**

# %%

```

```

from sklearn.tree import DecisionTreeClassifier

# %%
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
y_preds_dt = dt.predict(X_test)

# %%
clf_report_dt = classification_report(y_test, y_preds_dt, output_dict=True)
clf_report_dt

# %%
plot_probabilities(X_test, y_test, dt)

# %%
# Compute the confusion matrix
conf_matrix_dt = confusion_matrix(y_test, y_preds_dt)

# Display the confusion matrix
ConfusionMatrixDisplay(confusion_matrix=conf_matrix_dt).plot()
plt.show()

# %%
clf_report_dt

# %%
def parse_clf_report(clf_report):
    report = {}
    for key, value in clf_report.items():
        if key in y_test.unique().astype(str).tolist():
            report[key] = {k: v for k, v in value.items()}
    return report

# %%
lr_clf_df = pd.DataFrame(parse_clf_report(clf_report_lr)).T
lr_clf_df['model'] = 'Logistic Regression'
dt_clf_df = pd.DataFrame(parse_clf_report(clf_report_dt)).T
dt_clf_df['model'] = 'Decision Tree Classifier'

# %%
comparison_df = pd.concat([lr_clf_df, dt_clf_df], axis=0).reset_index()
comparison_df = comparison_df[['model', 'precision', 'recall', 'f1-score',
'support']]

# %%
comparison_df

# %% [markdown]
# ## **Part B**

# %%
from statsmodels.discrete.discrete_model import Probit

# %%

```

```

pt = Probit(y_train, X_train)
pt_res = pt.fit()
print(pt_res.summary())

# %%
y_preds_probs_pt = pt_res.predict(X_test)
y_preds_pt = (y_preds_probs_pt > 0.5).astype(int)

# %%
# Compute the confusion matrix
conf_matrix_pt = confusion_matrix(y_test, y_preds_pt)

# Display the confusion matrix
ConfusionMatrixDisplay(confusion_matrix=conf_matrix_pt).plot()
plt.show()

# %%
plot_probabilities(X_test, y_test, pt_res)

```

R Code

```

# Main Imports -----
# Set the base directory for the project
base_dir <- getwd()
setwd(base_dir)

# Define a helper function to install packages if not already installed
install <- function(pkg) {
  if (!require(pkg, character.only = TRUE)) {
    install.packages(pkg, dependencies = TRUE, quiet = TRUE)
  }
}

# Define a helper function to load packages
load_pkg <- function(pkg) {
  message(paste("Loading package:", pkg))
  library(pkg, character.only = TRUE, quietly = TRUE)
}

# Required packages for this analysis
pkgs <- c(
  "dplyr",
  "tidyr",
  "ggplot2",
  "caret",
  "e1071",
  "ROCR",
  "rpart",
  "rpart.plot",
  "MASS"
)

```

```

)

# Apply install and load functions
lapply(pkgs, install)
lapply(pkgs, load_pkg)

# Global File Config -----
datasets_dir <- "datasets"
datasets_path <- file.path(base_dir, datasets_dir)
file_name <- "Credit Card Defaulter Prediction.csv"
file_path <- file.path(datasets_path, file_name)

# Part A: Data Preprocessing and Model Training (Logistic Regression & Decision
Tree) -----

# Data Loading
df_a <- read.csv(file_path)

# Initial Data Inspection and Cleaning -----
cat("Initial Data Head:\n")
print(head(df_a))
cat("\nData Structure:\n")
str(df_a)

names(df_a) <- trimws(names(df_a))

cat("\nValue Counts for 'default' column:\n")
print(table(df_a$default))

cat("\nMissing Values per Column:\n")
print(colSums(is.na(df_a)))

df_a <- df_a %>% dplyr::select(-ID)

# Categorical Data Encoding -----
cat_data_cols <- names(df_a)[sapply(df_a, is.character)]
cat_data_cols

for (col in cat_data_cols) {
  df_a[[col]] <- as.numeric(factor(df_a[[col]])) - 1
}

cat("\nData Head after Encoding:\n")
print(head(df_a))

X <- df_a %>% dplyr::select(-default)
y <- as.factor(df_a$default)

# Data Splitting
set.seed(42)
train_index <- createDataPartition(y, p = 0.8, list = FALSE)
X_train <- X[train_index, ]
X_test <- X[-train_index, ]
y_train <- y[train_index]

```

```

y_test <- y[-train_index]

# Scaling Numerical Features (MinMaxScaler equivalent)
preproc_scaler <- preProcess(X_train, method = c("range"))
X_train_scaled <- predict(preproc_scaler, X_train)
X_test_scaled <- predict(preproc_scaler, X_test)

# Convert specific encoded categorical features back to factors for modeling
# This aligns with the instructor's practice for handling categorical predictors
in glm
categorical_cols_to_factor <- c("SEX", "EDUCATION", "MARRIAGE")

for (col in categorical_cols_to_factor) {
  if (col %in% names(X_train_scaled)) {
    X_train_scaled[[col]] <- as.factor(X_train_scaled[[col]])
  }
  if (col %in% names(X_test_scaled)) {
    X_test_scaled[[col]] <- as.factor(X_test_scaled[[col]])
  }
}

# Feature Selection using Correlation -----
# Calculate correlation between each feature in X_train_scaled and y_train
# Ensure y_train is numeric for correlation calculation
y_train_numeric_for_corr <- as.numeric(as.character(y_train))

# For correlation, we should use numeric columns only.
# Temporarily convert factor columns to numeric for correlation calculation.
X_train_scaled_numeric_for_corr <- X_train_scaled
for (col in categorical_cols_to_factor) {
  if (col %in% names(X_train_scaled_numeric_for_corr)) {
    X_train_scaled_numeric_for_corr[[col]] <-
as.numeric(as.character(X_train_scaled_numeric_for_corr[[col]]))
  }
}

# Initialize a data frame to store feature scores
feature_scores_df <- data.frame(
  feature = names(X_train_scaled_numeric_for_corr),
  feature_score = NA
)

# Calculate absolute correlation for each feature
for (i in seq_along(names(X_train_scaled_numeric_for_corr))) {
  feature_name <- names(X_train_scaled_numeric_for_corr)[i]
  # Use abs() for absolute correlation to consider strength regardless of
direction
  feature_scores_df$feature_score[i] <-
abs(cor(X_train_scaled_numeric_for_corr[[feature_name]],
y_train_numeric_for_corr))
}

feature_scores_df <- feature_scores_df %>%

```

```

    arrange(desc(feature_score))

cat("\nFeature Scores from Correlation:\n")
print(feature_scores_df)

# Select features with correlation score > 0.01 (this threshold can be adjusted)
selected_features <- feature_scores_df %>%
  filter(feature_score > 0.01) %>%
  pull(feature)

cat("\nSelected Features:\n")
print(selected_features)

X_train_selected <- X_train_scaled %>% dplyr::select(all_of(selected_features))
X_test_selected <- X_test_scaled %>% dplyr::select(all_of(selected_features))

# Utility Function for Plotting ROC Curve
plot_probabilities <- function(X_test, y_test, model, model_name = "Model") {
  if (inherits(model, "glm")) {
    pred_probs <- predict(model, newdata = as.data.frame(X_test), type =
"response")
  } else if (inherits(model, "rpart")) {
    pred_probs <- predict(model, newdata = as.data.frame(X_test), type = "prob")[,
"1"]
  } else if (inherits(model, "probit_model_MASS")) {
    pred_probs <- predict(model, newdata = as.data.frame(X_test), type =
"response")
  } else {
    stop("Unsupported model type for plot_probabilities.")
  }

  pred_roc <- prediction(pred_probs, y_test)
  perf_roc <- performance(pred_roc, "tpr", "fpr")
  auc_value <- as.numeric(performance(pred_roc, "auc")@y.values)

  plot(perf_roc,
    col = "blue",
    main = paste("Receiver Operating Characteristic (ROC) Curve |",
model_name),
    xlab = "False Positive Rate",
    ylab = "True Positive Rate")
  abline(a = 0, b = 1, col = "red", lty = 2)
  legend("bottomright", legend = paste("ROC curve (area =", round(auc_value, 2),
  ")),
    col = "blue", lty = 1, cex = 0.8)
}

# Logistic Regression Model -----
cat("\n--- Logistic Regression Model ---\n")
train_data_lr <- cbind(X_train_selected, default = y_train)

model_lr <- glm(default ~ ., data = train_data_lr, family = binomial(link =
"logit"))

```



```

y_preds_lr_prob <- predict(model_lr, newdata = X_test_selected, type = "response")
y_preds_lr_class <- factor(ifelse(y_preds_lr_prob > 0.5, 1, 0), levels =
levels(y_test))

conf_matrix_lr <- confusionMatrix(y_preds_lr_class, y_test)
cat("\nConfusion Matrix for Logistic Regression:\n")
print(conf_matrix_lr)
plot_probabilities(X_test_selected, y_test, model_lr, "Logistic Regression")

# Decision Tree Model -----
cat("\n--- Decision Tree Model ---\n")
train_data_dt <- cbind(X_train_selected, default = y_train)

model_dt <- rpart(default ~ ., data = train_data_dt, method = "class", control =
rpart.control(minsplit = 20, cp = 0.001))

y_preds_dt_prob <- predict(model_dt, newdata = X_test_selected, type = "prob")[,
"1"]
y_preds_dt_class <- factor(ifelse(y_preds_dt_prob > 0.5, 1, 0), levels =
levels(y_test))

conf_matrix_dt <- confusionMatrix(y_preds_dt_class, y_test)
cat("\nConfusion Matrix for Decision Tree:\n")
print(conf_matrix_dt)
plot_probabilities(X_test_selected, y_test, model_dt, "Decision Tree")

# Comparison of Models -----

# Function to parse and format classification report metrics
parse_clf_report <- function(conf_matrix, model_name, y_test_true) {
  # Extract the confusion matrix table
  table <- conf_matrix$table

  # Get class labels from the confusion matrix
  classes <- colnames(table)

  # Calculate metrics for each class
  metrics_list <- lapply(classes, function(cls) {
    if (cls == "0") {
      TP <- table["0", "0"] # True Positives (correctly predicted 0s)
      FN <- table["0", "1"] # False Negatives (0s predicted as 1s)
      FP <- table["1", "0"] # False Positives (1s predicted as 0s)
      TN <- table["1", "1"] # True Negatives (correctly predicted 0s)
    } else { # cls == "1"
      TP <- table["1", "1"] # True Positives (correctly predicted 1s)
      FN <- table["1", "0"] # False Negatives (1s predicted as 0s)
      FP <- table["0", "1"] # False Positives (0s predicted as 1s)
      TN <- table["0", "0"] # True Negatives (correctly predicted 0s)
    }
  })

  precision <- TP / (TP + FP)
  recall <- TP / (TP + FN)

```

```

f1_score <- 2 * (precision * recall) / (precision + recall)

# Handle NaN for cases where (TP + FP) or (TP + FN) is zero
if (is.nan(precision)) precision <- 0
if (is.nan(recall)) recall <- 0
if (is.nan(f1_score)) f1_score <- 0

support <- sum(y_test_true == cls)

data.frame(
  model = model_name,
  class = cls,
  precision = precision,
  recall = recall,
  f1_score = f1_score,
  support = support
)
})

# Combine results into a single data frame
do.call(rbind, metrics_list)
}

lr_clf_df <- parse_clf_report(conf_matrix_lr, "Logistic Regression", y_test)
dt_clf_df <- parse_clf_report(conf_matrix_dt, "Decision Tree Classifier", y_test)

comparison_df <- rbind(lr_clf_df, dt_clf_df)

cat("\n--- Model Comparison ---\n")
print(comparison_df)

# Part B: Probit Model -----
cat("\n--- Probit Regression Model ---\n")

y_train_numeric <- as.numeric(as.character(y_train))

train_data_pt <- cbind(X_train_selected, default = y_train_numeric)

model_pt <- glm(default ~ ., data = train_data_pt, family = binomial(link =
"probit"))
class(model_pt) <- c("probit_model_MASS", class(model_pt))

cat("\nProbit Regression Results Summary:\n")
print(summary(model_pt))

y_preds_probs_pt <- predict(model_pt, newdata = X_test_selected, type =
"response")
y_preds_pt_class <- factor(ifelse(y_preds_probs_pt > 0.5, 1, 0), levels =
levels(y_test))

conf_matrix_pt <- confusionMatrix(y_preds_pt_class, y_test)
cat("\nConfusion Matrix for Probit Model:\n")
print(conf_matrix_pt)

```

```
plot_probabilities(X_test_selected, y_test, model_pt, "Probit Model")
```